

## **Chapter 6: Coping with System Failures**

### **Assignment Questions**

- 1. Define backup and recovery. Compare full, incremental, and differential backups with example.**
- 2. Explain log-based recovery methods. Compare immediate and deferred updates with examples.**
- 3. Discuss the challenges of system failures and how RAID mitigates them. Provide examples of RAID configurations used in industry.**
- 4. What is shadow paging? Discuss its advantages and limitations with examples.**
- 5. Write a program to simulate checkpointing in a simple database.**
- 6. How does the Write-Ahead Logging (WAL) technique ensure durability in databases?**
- 7. Discuss the importance of hot backups in 24/7 systems like e-commerce platforms.**

## Chapter 7: Coping with System Failures (4 Hrs)

System failures can occur due to hardware malfunctions, software bugs, or external disasters, potentially leading to data loss or corruption. This chapter focuses on techniques and strategies for ensuring data durability and availability in the face of such failures.

### 1. Backup and Recovery Concepts (Expanded)

---

#### 1.1 Backup

##### Definition

- A systematic process of creating copies of database data to ensure data availability and protection from failure or corruption.
- 

##### Purpose

1. **Protection Against Data Loss:**
    - Guards against accidental deletion, hardware failures, and cyberattacks (e.g., ransomware).
  2. **Recovery from Failures:**
    - Ensures continuity after catastrophic events like power outages, server crashes, or natural disasters.
  3. **Data Archival:**
    - Used to store older data for compliance or long-term storage.
- 

#### Types of Backup

##### 1. Full Backup

- **Description:**
  - Creates a complete snapshot of the database.
  - Includes tables, indexes, logs, and metadata.
- **Advantages:**
  - Simplifies recovery as only one backup file is needed.
  - Serves as the foundation for incremental or differential backups.

- **Disadvantages:**
    - Time-consuming for large databases.
    - Requires significant storage space.
  - **Example:**
    - A company schedules full backups every Sunday at midnight during low-traffic hours.
- 

## 2. Incremental Backup

- **Description:**
    - Copies only the changes since the last backup (incremental or full).
  - **Advantages:**
    - Requires less storage space than full backups.
    - Faster backup process compared to full backups.
  - **Disadvantages:**
    - Restoration is slower because it requires applying the full backup and all subsequent incremental backups.
  - **Example:**
    - An e-commerce platform takes a full backup on Sunday and incremental backups daily to capture ongoing changes.
- 

## 3. Differential Backup

- **Description:**
  - Copies all changes since the last full backup.
- **Advantages:**
  - Faster recovery than incremental backups (requires only the full and latest differential backup).
  - Balances storage space and recovery time.
- **Disadvantages:**
  - Differential backup size grows over time as more changes accumulate.
  - Slower to back up compared to incremental backups.
- **Example:**

- A hospital database performs full backups on weekends and differential backups on weekdays to ensure fast recovery during emergencies.
- 

## **Other Backup Types**

### **1. Hot Backup:**

- Taken while the database is running and accessible.
- Example: An online banking system performs hot backups during active hours.

### **2. Cold Backup:**

- Taken while the database is offline.
- Example: A university shuts down its database at night to perform cold backups.

### **3. Logical Backup:**

- Exports database objects like schemas, tables, and data as scripts.
- Example: mysqldump in MySQL.

### **4. Physical Backup:**

- Copies raw database files at the storage level.
  - Example: Copying .dbf files in Oracle.
- 

## **Advantages of Backups**

### **1. Data Protection:**

- Prevents permanent loss of data due to failures or attacks.

### **2. Recovery Speed:**

- Enables quick restoration after disasters.

### **3. Legal Compliance:**

- Meets regulatory requirements for data retention.

### **4. Operational Continuity:**

- Minimizes downtime during failures.
- 

## **Disadvantages of Backups**

### **1. Storage Overhead:**

- Requires additional storage resources.
  - 2. **Time-Consuming:**
    - Full backups can take significant time for large databases.
  - 3. **Management Complexity:**
    - Requires proper scheduling, monitoring, and testing.
  - 4. **Performance Impact:**
    - Backup operations may affect live database performance during hot backups.
- 

## 1.2 Recovery

### Definition

- The process of restoring a database to its last consistent state after a failure, ensuring data integrity.
- 

### Phases of Recovery

#### 1. Analysis Phase

- Identifies the state of transactions at the time of failure.
  - Examines transaction logs to determine which transactions were committed, incomplete, or uncommitted.
- 

#### 2. Redo Phase

- Reapplies changes made by committed transactions to ensure all committed updates are reflected in the database.
  - **Example:**
    - A salary update for Employee A was committed but not written to disk due to a crash. The redo phase ensures this change is applied during recovery.
  - **Advantages:**
    - Ensures no committed transaction is lost.
  - **Disadvantages:**
    - Requires well-maintained logs for effective recovery.
-

### 3. Undo Phase

- Rolls back changes made by uncommitted transactions to restore the database to a consistent state.
  - **Example:**
    - A transaction to delete a record started but did not complete before the crash. The undo phase cancels this deletion.
  - **Advantages:**
    - Ensures consistency by removing incomplete changes.
  - **Disadvantages:**
    - May increase recovery time for databases with a high number of uncommitted transactions.
- 

## Recovery Techniques

### 1. Manual Recovery:

- Database administrators manually restore data using backups and logs.
- Example: A small business manually restores its database after a ransomware attack.

### 2. Automated Recovery:

- Modern DBMSs automate recovery using transaction logs and checkpoints.
  - Example: PostgreSQL uses Write-Ahead Logging (WAL) to automate crash recovery.
- 

## Advantages of Recovery

### 1. Ensures Data Integrity:

- Restores the database to a consistent state.

### 2. Minimizes Downtime:

- Automated recovery quickly resumes database operations.

### 3. Handles Complex Failures:

- Deals with crashes, disk failures, and incomplete transactions.
- 

## Disadvantages of Recovery

- 1. **Dependency on Logs and Backups:**
  - Recovery is impossible without reliable backups or complete transaction logs.
- 2. **Time-Consuming:**
  - Recovery from large transaction logs or multiple backups can be slow.
- 3. **Complexity:**
  - Managing logs and backups requires expertise.

---

**Examples of Recovery Scenarios**

- 1. **System Crash:**
  - The DBMS restarts, analyzes logs, and performs redo/undo as needed.
  - Example: After a power failure, an e-commerce platform uses redo logging to ensure all completed orders are processed.
- 2. **Media Failure:**
  - Backups are restored to replace corrupted or lost files.
  - Example: A bank restores its database using the most recent full backup after a hard drive failure.
- 3. **Human Error:**
  - Accidentally deleted rows are restored using logical backups.
  - Example: A hospital recovers accidentally deleted patient records using a backup.

---

**Comparison of Backup and Recovery Techniques**

| Aspect           | Full Backup      | Incremental Backup             | Differential Backup   |
|------------------|------------------|--------------------------------|-----------------------|
| Backup Time      | Long             | Short                          | Moderate              |
| Restore Time     | Short            | Long (requires multiple files) | Moderate              |
| Storage Required | High             | Low                            | Moderate              |
| Best For         | Baseline backups | Daily changes                  | Faster recovery needs |

---

**Key Takeaways**

- **Backup:**

- Critical for protecting against data loss and ensuring business continuity.
- Full, incremental, and differential backups offer varying trade-offs between time, storage, and recovery complexity.
- **Recovery:**
  - Ensures consistency using **analysis**, **redo**, and **undo** phases.
  - Modern recovery techniques rely on transaction logs, checkpoints, and automated tools to minimize downtime.

## 2. Various Types of Backup (Expanded)

---

### 2.1 Hot Backup

- **Definition:**
    - A backup taken while the database is running and accessible to users. The database remains operational during the backup process.
  - **Key Features:**
    - Supports live systems without downtime.
    - Requires transaction logs to ensure the consistency of the data being backed up.
    - Often used in **24/7 operational environments**.
  - **Advantages:**
    1. No disruption to users or applications.
    2. Suitable for mission-critical systems requiring high availability.
    3. Transaction logs ensure data consistency.
  - **Disadvantages:**
    1. Requires additional resources to manage live backup processes.
    2. May involve complex configurations for maintaining consistency.
    3. Slower performance during backup due to increased workload on the system.
  - **Example:**
    - An online banking system performs a hot backup during business hours while users continue to access their accounts.
-



## 2.2 Cold Backup

- **Definition:**

- A backup taken when the database is offline and inaccessible to users. It ensures a consistent snapshot of the database.

- **Key Features:**

- Requires shutting down the database.
- No need for transaction logging as the database is static during the backup.

- **Advantages:**

1. Ensures data consistency without additional logging overhead.
2. Simple to implement compared to hot backups.
3. Suitable for small systems or during maintenance windows.

4. **Disadvantages:**

5. Requires system downtime, impacting availability.
6. Not feasible for systems that need to operate continuously.

- **Example:**

- A university database shuts down during the night to perform a cold backup.
- 

## 2.3 Online and Offline Backup

### Online Backup

- **Definition:**

- Similar to a hot backup, taken while the database is operational and accessible.

- **Key Features:**

- Involves maintaining transaction logs to ensure consistency.
- Often used for cloud-based and distributed systems.

### Offline Backup

- **Definition:**

- Similar to a cold backup, taken when the database is completely shut down.

- **Key Features:**

- Simplifies the backup process by avoiding transaction activity during the backup.

## Comparison

| Aspect       | Online Backup                 | Offline Backup                       |
|--------------|-------------------------------|--------------------------------------|
| Availability | Database remains operational. | Database is unavailable.             |
| Consistency  | Relies on transaction logs.   | Ensures consistency naturally.       |
| Use Case     | 24/7 systems (e.g., banking). | Low-demand systems (e.g., archives). |

---

## 2.4 Logical and Physical Backup

### Logical Backup

- **Definition:**

- Exports database schema, objects (tables, indexes, etc.), and data in formats like SQL scripts.

- **Key Features:**

- Independent of the underlying database system.
- Used for migrations, upgrades, or small database backups.

- **Advantages:**

1. Platform-independent: Can be restored on different DBMS systems.
2. Easy to modify or analyze as data is stored in readable formats.

- **Disadvantages:**

1. Slower to back up and restore large datasets.
2. Does not include database configuration files or logs.

- **Example:**

- **MySQL:**

```
mysqldump -u root -p database_name > backup.sql
```

---

### Physical Backup

- **Definition:**

- Copies raw database files (data, logs, configuration) at the storage level.

- **Key Features:**

- Includes the entire database structure, logs, and metadata.
- Faster backup and restore for large databases compared to logical backups.

- **Advantages:**

1. Faster than logical backups for large databases.
  2. Ensures consistency at the file system level.
- **Disadvantages:**
    1. Platform-dependent: Cannot be restored across different DBMS systems.
    2. Requires more storage compared to logical backups.

- **Example:**

- Backing up .dbf files in Oracle:

- ```
cp /path/to/database/files/* /path/to/backup/
```

---

## Other Backup Methods

### 1. Incremental and Differential Backups:

- Often combined with hot or cold backups to balance performance and recovery time.
  - Example: Incremental backups are common for large, continuously updated systems.

### 2. Snapshot Backups:

- Create point-in-time snapshots of the database using storage-level or database-level tools.
  - Suitable for cloud environments where storage snapshots are quick and efficient.

### 3. Cloud Backups:

- Backups stored in the cloud for redundancy and disaster recovery.
    - Example: Amazon RDS provides automated and manual backups for relational databases.
- 

## Comparison of Backup Types

| Aspect       | Hot Backup        | Cold Backup           | Logical Backup           | Physical Backup           |
|--------------|-------------------|-----------------------|--------------------------|---------------------------|
| Availability | No downtime.      | Requires downtime.    | No downtime.             | Depends on approach.      |
| Consistency  | Requires logging. | Naturally consistent. | Ensured through exports. | Consistent at file level. |

|                    |                            |                            |                             |                          |
|--------------------|----------------------------|----------------------------|-----------------------------|--------------------------|
| <b>Performance</b> | Affects live performance.  | No impact during downtime. | Slower for large datasets.  | Fast for large datasets. |
| <b>Use Case</b>    | High-availability systems. | Archival or maintenance.   | Small datasets, migrations. | Large, static datasets.  |

---

### Summary

- **Hot Backup:** Best for high-availability systems like e-commerce platforms and banks.
- **Cold Backup:** Ideal for low-demand systems or during maintenance windows.
- **Logical Backup:** Used for migrations and readable backups.
- **Physical Backup:** Suited for large datasets with platform dependency.

Each backup type has unique trade-offs in terms of availability, consistency, performance, and use cases. Organizations should choose the method(s) that best align with their operational needs and disaster recovery strategies.

## 3. Issues and Models for Resilient Operation

---

### Challenges in Resilience

#### 1. Data Corruption

- **Definition:**
  - Data corruption occurs when information becomes unreadable, incomplete, or inaccurate due to hardware failures, software bugs, or power outages.
- **Causes:**
  1. Sudden power failures during write operations.
  2. Disk errors or bad sectors.
  3. Software bugs in database or storage management.
  4. Malware or ransomware attacks.
- **Impact:**
  - Corruption can render parts of the database unusable or lead to inaccurate query results.
- **Prevention:**
  1. **Transaction Logging:**

- Use Write-Ahead Logging (WAL) to ensure committed transactions are recorded before making changes.
  - 2. **Checksum Mechanisms:**
    - Verify data integrity during read and write operations.
  - 3. **Fault-Tolerant Hardware:**
    - Deploy RAID configurations to prevent single-disk failures.
  - 4. **Replication:**
    - Maintain multiple copies of the database to recover corrupted data from a healthy replica.
- 

## 2. Performance Overhead

- **Definition:**
    - Backup and recovery processes consume system resources, potentially impacting normal database operations.
  - **Causes:**
    1. Frequent full backups during peak hours.
    2. Resource-intensive recovery operations (e.g., scanning logs, reapplying changes).
    3. High I/O activity during replication or synchronization.
  - **Impact:**
    - Slows down query performance, affecting user experience or operational efficiency.
  - **Mitigation Strategies:**
    1. **Schedule Backups During Off-Peak Hours:**
      - Avoid performing backups when the system is under heavy load.
    2. **Incremental or Differential Backups:**
      - Reduce the time and resources needed for backup processes.
    3. **Throttling:**
      - Limit the resource consumption of backup tasks to prioritize active queries.
- 

## 3. Storage Management

- **Definition:**

- Efficiently managing storage resources for backups, logs, and replication.
  - **Challenges:**
    1. **Storage Space:**
      - Frequent backups and large transaction logs require significant storage space.
    2. **Data Retention Policies:**
      - Determining how long backups and logs should be retained.
    3. **Fragmentation:**
      - Managing storage fragmentation as data grows and changes.
  - **Solutions:**
    1. **Compression:**
      - Compress backups to reduce storage requirements.
    2. **Cloud Storage:**
      - Use scalable cloud solutions to store backups.
    3. **Retention Policies:**
      - Implement policies to archive or delete outdated backups and logs.
- 

## Models for Resilience

### 1. RAID Storage

- **Definition:**
  - RAID (Redundant Array of Independent Disks) is a storage technology that combines multiple physical disks into a logical unit to provide fault tolerance and improved performance.
- **RAID Levels:**
  1. **RAID 1 (Mirroring):**
    - Duplicates data across two or more disks.
    - Ensures redundancy and fault tolerance.
    - **Example:** Even if one disk fails, the data is still available on the mirrored disk.
  2. **RAID 5 (Striping with Parity):**
    - Distributes data and parity across multiple disks.

- **Advantage:** Balances performance and fault tolerance.
- **Disadvantage:** Slower write performance due to parity calculations.

### 3. **RAID 10 (Combination of RAID 1 and 0):**

- Combines mirroring and striping for high redundancy and performance.
  - **Advantages:**
    - Protects against hardware failures.
    - Improves read and write performance (depending on RAID level).
  - **Disadvantages:**
    - Higher hardware costs.
    - Requires additional configuration and management.
- 

## 2. Replication

- **Definition:**
  - The process of maintaining a real-time copy of the database on a separate server.
- **Types of Replication:**

### 1. **Synchronous Replication:**

- Changes are applied to both the primary and replica database simultaneously.
- **Advantage:** Ensures data consistency.
- **Disadvantage:** Slower performance due to increased latency.

### 2. **Asynchronous Replication:**

- Changes are applied to the replica database after a delay.
- **Advantage:** Faster write operations on the primary database.
- **Disadvantage:** May result in data loss if the primary database fails before synchronization.
- **Use Cases:**
  - Disaster recovery: Ensures a backup copy is available for failover.
  - Load balancing: Distributes read workloads across replicas to improve performance.
- **Advantages:**

- High availability and fault tolerance.
  - Enables distributed systems for improved performance.
  - **Disadvantages:**
    - Requires additional hardware and network bandwidth.
    - Increased complexity for managing replicas.
- 

### 3. Disaster Recovery Plans

- **Definition:**
    - A comprehensive strategy to restore database operations after catastrophic failures.
  - **Key Components:**
    1. **Risk Assessment:**
      - Identify potential risks, such as hardware failures, cyberattacks, or natural disasters.
    2. **Data Prioritization:**
      - Identify critical systems and data that need immediate recovery.
    3. **Regular Testing:**
      - Periodically test disaster recovery procedures to ensure readiness.
  - **Steps in Disaster Recovery:**
    1. Backup restoration from the most recent backup.
    2. Apply transaction logs to recover the most recent state.
    3. Failover to a replicated system if available.
  - **Advantages:**
    - Minimizes downtime during major failures.
    - Ensures business continuity and data availability.
  - **Disadvantages:**
    - High cost of implementation and testing.
    - Requires significant planning and resources.
- 

### Comparison of Resilience Models



| Aspect              | RAID Storage                    | Replication                    | Disaster Recovery Plans            |
|---------------------|---------------------------------|--------------------------------|------------------------------------|
| Focus               | Hardware-level fault tolerance. | Real-time data duplication.    | Overall system recovery.           |
| Implementation Cost | Moderate to high.               | High (additional servers).     | High (involves extensive testing). |
| Performance Impact  | Improves read/write (RAID 0).   | Slows write operations (sync). | No direct impact (proactive).      |
| Use Case            | Disk failure protection.        | High availability.             | Catastrophic failure recovery.     |

---

### Summary

- **Challenges in Resilience:**
  - **Data Corruption:** Prevent using transaction logs and checksums.
  - **Performance Overhead:** Mitigate with optimized backup schedules and efficient methods like incremental backups.
  - **Storage Management:** Use compression, cloud storage, and retention policies to handle backup sizes.
- **Models for Resilience:**
  - **RAID Storage:** Protects against disk failures and improves performance.
  - **Replication:** Ensures high availability with real-time copies.
  - **Disaster Recovery Plans:** Provides a systematic approach to restoring operations after catastrophic failures.

A combination of these models ensures robust and resilient database operations, minimizing downtime and data loss.

### 4. Undo/Redo Logging

Undo and redo logging are essential mechanisms in database management systems (DBMS) to ensure the **atomicity** and **durability** properties of transactions, especially during crash recovery. These techniques help the DBMS recover from system failures and maintain a consistent database state.

---

### Undo Logging

## Purpose

- Undo logging is used to **reverse the changes** made by incomplete or uncommitted transactions during a system crash or failure.
- It ensures that no partial changes from failed transactions remain in the database, maintaining consistency.

## Steps in Undo Logging

### 1. Identify Uncommitted Transactions:

- During recovery, the system scans the log to find transactions that were active but not committed at the time of the crash.

### 2. Undo Changes:

- For each uncommitted transaction, reverse its changes in reverse order (last operation undone first).
- Changes are reverted using the **before-image** of the data stored in the log.

### 3. Restore to Previous State:

- Ensure the database is restored to the state before the transaction began.

## Requirements for Undo Logging

### 1. Write-Ahead Logging (WAL):

- Logs must be written to disk **before** the corresponding data changes are applied to the database.

### 2. Log Entries:

- Include a **before-image** (original value of the data before the change).

## Example of Undo Logging

- **Transaction:** Update a salary field:

**UPDATE Employee SET Salary = 50000 WHERE EmpID = 101;**

- **Log Entry** (Before Image):

**<T1, EmpID=101, Salary=40000> -- Before update**

- **Crash Scenario:**

- If the transaction is not committed before the crash, undo logging restores the salary to its original value (40000).

---

## Redo Logging

## Purpose

- Redo logging ensures that changes made by **committed transactions** are **reapplied** to the database, even if those changes were not written to disk before the crash.
- This guarantees the **durability** of committed transactions.

## Steps in Redo Logging

### 1. Identify Committed Transactions:

- During recovery, the system scans the log to find transactions marked as committed.

### 2. Redo Changes:

- For each committed transaction, reapply its changes in the same order as they were made.
- Changes are reapplied using the **after-image** stored in the log.

## Requirements for Redo Logging

### 1. Write-Ahead Logging (WAL):

- Logs must be written to disk before a transaction is marked as committed.

### 2. Log Entries:

- Include an **after-image** (new value of the data after the change).

## Example of Redo Logging

- **Transaction:** Insert a new record:

**INSERT INTO Employee (EmpID, Name, Salary) VALUES (102, 'John', 45000);**

- **Log Entry** (After Image):

**<T2, EmpID=102, Name='John', Salary=45000> -- After insert**

- **Crash Scenario:**

- If the transaction was committed but not written to disk, redo logging ensures the insertion is reapplied.

---

## Combined Undo/Redo Logging

Most modern database systems use both undo and redo logging to handle recovery efficiently. This combination ensures:

### 1. Undo for Uncommitted Transactions:

- Ensures partial changes are reversed.

## 2. Redo for Committed Transactions:

- Ensures all committed changes are preserved and reapplied if necessary.

---

### Steps in Combined Undo/Redo Logging

#### 1. Write-Ahead Logging (WAL):

- Ensure that all log entries (undo and redo) are written to disk before the actual changes are applied.

#### 2. Crash Recovery:

- **Undo Phase:**
  - Identify uncommitted transactions and revert their changes using undo logs.
- **Redo Phase:**
  - Identify committed transactions and reapply their changes using redo logs.

---

### Comparison of Undo and Redo Logging

| Aspect                    | Undo Logging                                   | Redo Logging                                   |
|---------------------------|------------------------------------------------|------------------------------------------------|
| <b>Purpose</b>            | Revert uncommitted changes.                    | Reapply committed changes.                     |
| <b>Log Entry</b>          | Stores <b>before-image</b> of data.            | Stores <b>after-image</b> of data.             |
| <b>Order of Execution</b> | Changes undone in <b>reverse order</b> .       | Changes redone in <b>original order</b> .      |
| <b>Primary Use Case</b>   | Ensures consistency after failed transactions. | Ensures durability for committed transactions. |
| <b>Recovery Action</b>    | Rollback uncommitted transactions.             | Reapply committed changes not yet persisted.   |

---

### Advantages of Undo/Redo Logging

#### Undo Logging

- Ensures database consistency by reversing incomplete changes.
- Simple to implement for systems with low write activity.

#### Redo Logging

- Ensures committed transactions are durable, even if not flushed to disk.
- Critical for high-availability systems requiring guaranteed durability.

#### Combined Logging

- Handles both incomplete and committed transactions in a single recovery process.

- Ensures full compliance with the ACID properties (Atomicity, Consistency, Isolation, Durability).
- 

### Disadvantages of Undo/Redo Logging

#### 1. Performance Overhead:

- Write-ahead logging increases disk I/O as logs must be written before actual changes.

#### 2. Complexity:

- Combined undo/redo logging requires careful implementation to manage logs and recovery efficiently.

#### 3. Storage Requirements:

- Logs can grow large, requiring periodic pruning or archiving.
- 

### Real-World Example of Combined Logging

#### • Scenario:

- A financial application records transactions in a database. During a power failure:
  - A salary update (T1) was uncommitted.
  - A bonus addition (T2) was committed but not flushed to disk.

#### • Recovery Process:

- **Undo Phase:**
    - Revert the uncommitted salary update using the undo log.
  - **Redo Phase:**
    - Reapply the committed bonus addition using the redo log.
- 

### Key Takeaways

- **Undo Logging** ensures incomplete transactions are reversed, maintaining database consistency.
- **Redo Logging** guarantees that committed transactions are preserved, even after a crash.
- Combined logging systems, used in modern databases, efficiently handle crash recovery, supporting both undo and redo operations.

## 5. Protection Against Media Failures

### Definition

Media failures refer to failures of the physical storage medium, such as hard drives, solid-state drives, or other storage devices. These failures can result in partial or complete data loss if the affected data is not recoverable. Media failures are typically caused by hardware malfunctions, physical damage, power surges, or aging storage devices.

---

### Techniques for Protection Against Media Failures

#### 1. Data Redundancy

- **Description:**
  - Data redundancy involves storing duplicate copies of data to ensure availability and recoverability in the event of media failure.
- **Methods:**

##### 1. RAID (Redundant Array of Independent Disks):

- Combines multiple physical disks into logical units for fault tolerance and performance.
- **Common RAID Levels:**
  - **RAID 0 (Striping)** : Striping data file across two or more disks for faster access
  - **RAID 1 (Mirroring)**: Duplicates data across two disks; one disk failure does not lead to data loss.
  - **RAID 5 (Striping with Parity)**: Distributes data and parity across disks; one disk failure can be recovered using parity data.
  - **RAID 6 (Double Parity)**: Protects against simultaneous failures of up to two disks.
  - **RAID 10 (Mirroring and Striping)**: Combines mirroring and striping for high performance and fault tolerance.

##### 2. Replication:

- Creates real-time or scheduled copies of data on different physical storage systems.
- **Synchronous Replication**: Ensures both primary and replica storage have identical data at all times.

- **Asynchronous Replication:** Updates replicas after a delay, reducing performance impact on primary storage.
  - **Advantages:**
    - Prevents single points of failure.
    - Reduces downtime by enabling quick failover.
  - **Disadvantages:**
    - Requires additional hardware and storage space.
    - RAID cannot protect against logical corruption (e.g., accidental deletion).
- 

## 2. Regular Backups

- **Description:**
  - Backups are scheduled to create copies of data that can be restored in the event of failure.
- **Methods:**
  - **Full Backups:**
    - Periodically back up the entire database or system.
    - Ensures a complete, restorable dataset.
  - **Incremental Backups:**
    - Backs up only the changes made since the last backup.
    - Reduces time and storage requirements.
  - **Differential Backups:**
    - Backs up changes made since the last full backup.
    - Balances time and space requirements.
  - **Cloud Backups:**
    - Stores backups in remote cloud storage for added resilience against local failures.
- **Best Practices:**
  1. Schedule backups during low-usage periods to reduce performance impact.
  2. Use **offsite backups** to protect against physical damage to the data center (e.g., due to fire or flooding).

3. Regularly test backups to ensure data integrity and successful restoration.

- **Advantages:**

- Provides recovery options for catastrophic failures.
- Safeguards against logical errors and accidental deletions.

- **Disadvantages:**

- Time-consuming for large datasets.
  - Restoring from backups may involve some downtime.
- 

### 3. Storage Monitoring

- **Description:**

- Actively monitoring storage devices for signs of impending failure allows preemptive action to prevent data loss.

- **Methods:**

1. **S.M.A.R.T. (Self-Monitoring, Analysis, and Reporting Technology):**

- Monitors disk health by analyzing attributes like temperature, read/write errors, and spin-up time.
- Provides warnings when disks are at risk of failure.

2. **Predictive Analysis Tools:**

- Uses machine learning and advanced analytics to predict failures based on historical trends and real-time data.
- Example: Tools like Dell OpenManage or AWS Elastic Block Store (EBS) Health Monitoring.

3. **Proactive Alerts:**

- Configure alerts for signs of degradation, such as increased I/O latency or bad sectors.

- **Best Practices:**

1. Replace storage devices showing signs of wear or nearing end-of-life.
2. Use redundant systems (e.g., RAID) to avoid immediate failure impact.

- **Advantages:**

- Minimizes unplanned downtime by predicting and addressing failures proactively.
- Enhances storage reliability and lifespan.

- **Disadvantages:**



- Monitoring tools may require additional configuration and resources.
  - Predictions are not always accurate.
- 

## **Additional Techniques for Media Failure Protection**

### **4. Snapshot Backups**

- **Description:**
    - Snapshots create point-in-time copies of data, allowing quick restoration to a specific moment.
  - **Advantages:**
    - Fast recovery and minimal storage requirements (incremental snapshots).
    - Useful for undoing accidental changes or corruption.
  - **Example:**
    - A cloud provider like AWS offers snapshot-based backups for Elastic Block Store (EBS) volumes.
- 

### **5. Cloud Storage and Replication**

- **Description:**
    - Stores data in cloud services to provide additional resilience against local media failures.
  - **Advantages:**
    - High durability, with cloud providers offering multiple copies across geographically distributed data centers.
    - Simplifies disaster recovery and ensures access from anywhere.
  - **Example:**
    - Google Cloud Storage or Amazon S3 for regular backups.
- 

### **6. Disk Scrubbing**

- **Description:**
  - Periodically checks and repairs data inconsistencies or corruption on storage devices.

- **Use Case:**
    - Common in RAID systems to verify parity and ensure data integrity.
- 

## **Advantages of Media Failure Protection**

1. **Minimized Data Loss:**
    - Techniques like RAID, replication, and regular backups reduce the risk of losing critical data.
  2. **Increased Availability:**
    - Redundancy and replication ensure data is accessible even if a storage device fails.
  3. **Proactive Recovery:**
    - Monitoring and predictive tools provide early warnings, enabling proactive actions.
  4. **Scalability:**
    - Cloud storage and replication allow businesses to scale protection as data grows.
- 

## **Disadvantages of Media Failure Protection**

1. **Cost Overhead:**
    - Additional storage devices, RAID setups, and cloud services can be expensive.
  2. **Complexity:**
    - Setting up and managing redundancy, monitoring tools, and backups requires expertise.
  3. **Performance Impact:**
    - RAID and replication may slow down write operations due to parity calculations or synchronization.
- 

## **Real-World Examples of Media Failure Protection**

1. **E-Commerce Platform:**
  - Uses RAID 10 for local redundancy and asynchronous replication to a remote data center to ensure uptime during disk failures.
2. **Healthcare System:**

- Implements daily incremental backups and weekly full backups to protect patient records against hardware failures.

### 3. Cloud-Based Startups:

- Rely on Amazon S3 with versioning enabled for backup and recovery, providing multi-region durability.

---

## Key Takeaways

- **Data Redundancy:** Use RAID and replication to minimize downtime and data loss during storage failures.
- **Regular Backups:** Schedule backups and store them offsite or in the cloud for added resilience.
- **Storage Monitoring:** Monitor disk health to predict and address failures before they occur.

A comprehensive strategy combining these techniques ensures robust protection against media failures, enabling quick recovery and business continuity.

## 6. Log-Based Recovery Methods

---

### Definition

Log-based recovery methods rely on **transaction logs**, which are sequential records of database operations. These logs serve as a reliable source to recover the database after a failure, ensuring consistency, atomicity, and durability as per the ACID properties of transactions.

---

### Steps in Log-Based Recovery

#### 1. Maintain Transaction Logs

- **Description:**
  - Transaction logs record every database operation, including updates, inserts, and deletes.
  - Each log entry typically contains:
    - **Transaction ID:** Identifies the transaction.
    - **Before-Image:** The value of the data before the operation.
    - **After-Image:** The value of the data after the operation.

- Example:
  - If a transaction updates a row, the log might look like:

**<T1, Before: Salary=50000, After: Salary=55000>**

- **Purpose:**

- Logs provide a complete history of changes, enabling recovery in case of a crash.
  - They ensure that no transaction is lost, even if the database is interrupted.
- 

## 2. Identify Transactions

- **Description:**

- During recovery, the database system scans the transaction logs to identify:
  1. **Committed Transactions:**
    - Transactions that were completed and marked as committed before the crash.
  2. **Uncommitted Transactions:**
    - Transactions that were incomplete or not committed at the time of failure.

- **How It Works:**

- The DBMS reads the log records to identify the transaction state.
- Example:

**<T1 BEGIN>**

**<T1 UPDATE Salary=50000 TO 55000>**

**<T1 COMMIT>**

- **T1** is a committed transaction.

**<T2 BEGIN>**

**<T2 DELETE EmployeeID=101>**

- **T2** is uncommitted and needs to be undone.
- 

## 3. Redo and Undo

- **Redo:**

- **Purpose:** Reapplies changes made by committed transactions to ensure durability.
- **Process:**
  - For each committed transaction, reapply its changes using the **after-image** stored in the log.
- **Example:**
  - Transaction T1:
 

**<T1, Before: Salary=50000, After: Salary=55000>**

    - Redo: Update Salary to 55000 to ensure the change is applied.
- **Undo:**
  - **Purpose:** Reverts changes made by uncommitted transactions to maintain consistency.
  - **Process:**
    - For each uncommitted transaction, reverse its changes using the **before-image** stored in the log.
  - **Example:**
    - Transaction T2:
 

**<T2, Before: EmployeeID=101, After: Deleted>**

      - Undo: Restore EmployeeID=101 to its previous state.

#### 4. Write-Ahead Logging (WAL)

- **Description:**
  - A critical rule for log-based recovery, **WAL** ensures that logs are written to disk **before** the actual database changes are applied.
- **Purpose:**
  - Guarantees that all changes are recorded in the logs, even if the system crashes before completing the database update.
- **Implementation:**
  - For every operation:
    1. Write a log entry to the disk.
    2. Apply the operation to the database.

- Example:
    - A transaction to update a salary logs:  
**<T3, Before: Salary=45000, After: Salary=50000>**
    - Only after the log is written, the Salary column is updated in the database.
- 

## Types of Log-Based Recovery

### 1. Undo Logging

- Focuses on rolling back changes made by uncommitted transactions.
  - **Log Entry:**
    - Records the **before-image** of the data.
  - **Steps:**
    1. Identify uncommitted transactions.
    2. Revert changes using the before-image.
  - **Example:**
    - Transaction T4:  
**<T4, Before: Name='John', After: Name='Jonathan'>**
      - Undo: Restore Name to John.
- 

### 2. Redo Logging

- Focuses on reapplying changes made by committed transactions.
- **Log Entry:**
  - Records the **after-image** of the data.
- **Steps:**
  1. Identify committed transactions.
  2. Reapply changes using the after-image.
- **Example:**
  - Transaction T5:  
**<T5, Before: Age=30, After: Age=31>**
    - Redo: Update Age to 31.

---

### 3. Combined Undo/Redo Logging

- Modern databases use both undo and redo logging for efficient recovery.
- **Log Entry:**
  - Records both before-image and after-image.
- **Steps:**
  1. Undo changes for uncommitted transactions.
  2. Redo changes for committed transactions.

---

### Advantages of Log-Based Recovery

1. **Data Integrity:**
  - Ensures that the database remains consistent even after a crash.
2. **Durability:**
  - Guarantees that committed transactions are not lost.
3. **Efficient Recovery:**
  - Allows targeted recovery of affected transactions.
4. **ACID Compliance:**
  - Fully supports atomicity, consistency, isolation, and durability.

---

### Disadvantages of Log-Based Recovery

1. **Performance Overhead:**
  - Logging every transaction increases disk I/O, impacting performance.
2. **Storage Requirements:**
  - Logs can grow large, requiring regular maintenance.
3. **Complexity:**
  - Recovery algorithms can be complex to implement and manage.

---

### Real-World Applications

1. **Banking Systems:**

- Ensure that committed transactions (e.g., fund transfers) are durable, and incomplete ones are rolled back after a crash.
- Use WAL for transaction logs.

## 2. E-Commerce Platforms:

- Maintain logs to recover incomplete orders or payments in case of failures.
- Redo logging ensures all successful transactions are preserved.

## 3. Cloud Databases:

- Distributed systems like AWS RDS use log-based recovery to ensure data consistency across nodes during failovers.

---

### Key Takeaways

- **Logs** are the backbone of recovery mechanisms, recording transaction details to handle crashes.
- **Undo Logging** reverts changes made by uncommitted transactions.
- **Redo Logging** reapplies changes made by committed transactions.
- **Write-Ahead Logging (WAL)** ensures logs are written before applying database changes.
- Combined undo/redo logging is the standard in modern DBMSs for efficient and reliable recovery.

## 7. Checkpoints

---

### Definition

A checkpoint is a mechanism in database management systems (DBMS) that marks a specific point in the transaction log, ensuring that all changes up to that point are saved to disk. Checkpoints help reduce the amount of log data that needs to be processed during recovery, significantly speeding up the process.

---

### Purpose

- **Speeds Up Recovery:**
  - Limits the number of log records that must be scanned and processed during crash recovery by establishing a consistent database state up to the checkpoint.
- **Ensures Consistency:**



- Writes all pending changes (dirty pages) from memory to disk, ensuring that the database reflects all committed transactions up to the checkpoint.
  - **Improves Efficiency:**
    - Reduces the time and effort required to redo or undo transactions after a failure.
- 

## Checkpoint Process

The checkpoint process ensures that the database is in a consistent state up to a specific point. The steps are as follows:

### 1. Flush Dirty Pages to Disk:

- Write all modified pages in the buffer pool (dirty pages) to persistent storage.
- Ensures that changes made by committed transactions are saved to disk.

### 2. Write a Checkpoint Record to the Log:

- Record a checkpoint marker in the transaction log.
- This marker indicates that all changes before this point are reflected on disk.

### 3. Reduce Log Entries for Recovery:

- The database no longer needs to process log records before the checkpoint during recovery.
  - Only logs after the checkpoint are needed for redo/undo operations.
- 

## Advantages

### 1. Faster Recovery:

- By limiting the logs that need to be processed, checkpoints significantly reduce recovery time after a crash.
- Example: If a checkpoint is taken every 10 minutes, only logs from the last 10 minutes are processed during recovery.

### 2. Reduced Log Size:

- Log entries before the checkpoint can be safely archived or discarded, freeing up storage space.

### 3. Consistency Assurance:

- Guarantees that all committed transactions before the checkpoint are permanently stored on disk.

#### 4. Improved System Stability:

- Regular checkpoints provide intermediate recovery points, minimizing data loss in case of failures.
- 

### Disadvantages

#### 1. Performance Overhead:

- Writing dirty pages and logs to disk during a checkpoint can temporarily slow down the system.
- May increase I/O load, impacting active transactions.

#### 2. Configuration Complexity:

- Determining the optimal frequency for checkpoints requires careful analysis of system workload and recovery requirements.
- Too frequent checkpoints increase overhead; infrequent checkpoints slow recovery.

#### 3. Temporary Blocking:

- Some DBMS implementations may pause transaction processing during a checkpoint.
- 

### Checkpoint Frequency

The frequency of checkpoints impacts both performance and recovery time:

- **Frequent Checkpoints:**

- Reduce recovery time but increase system overhead.
- Suitable for high-availability systems requiring fast recovery.

- **Infrequent Checkpoints:**

- Lower performance overhead but increase recovery time.
  - Suitable for systems with low transaction volumes.
- 

### Example of Checkpoint Implementation

#### Scenario:

- A banking system processes multiple transactions every second.
- To ensure consistency and quick recovery, checkpoints are taken every 5 minutes.

### Process:

1. Flush all transaction changes from memory to disk.
2. Write a checkpoint record:

**CHECKPOINT ID=12345 TIMESTAMP=2024-11-24 10:00:00**

3. During a system crash, the recovery process starts from this checkpoint, ignoring earlier log records.

---

### Real-World Applications

#### 1. Relational Databases:

- Oracle, MySQL, and PostgreSQL use checkpoints to minimize recovery times.
- Example: PostgreSQL implements periodic checkpoints to ensure fast recovery after crashes.

#### 2. High-Transaction Systems:

- E-commerce platforms use frequent checkpoints to prevent data loss and enable quick failover during crashes.

#### 3. Distributed Databases:

- Systems like Apache Cassandra use checkpoints to maintain consistency across nodes in distributed environments.

---

### Key Takeaways

- **Definition:** Checkpoints mark a consistent state of the database in the transaction log.
- **Purpose:** Speeds up recovery by limiting log processing and reducing recovery time.
- **Process:** Flush dirty pages, write checkpoint markers, and truncate logs.
- **Advantages:**
  - Faster recovery.
  - Reduced log size and improved consistency.
- **Challenges:**
  - Performance overhead and configuration complexity.

By balancing checkpoint frequency and system workload, DBMSs can ensure efficient recovery while maintaining optimal performance.

## 8. Shadow Paging Technique

---

### Definition

Shadow paging is a crash recovery mechanism used in database systems. It maintains two versions of database pages:

1. **Current Page:** The active version where changes are made.
2. **Shadow Page:** A stable, unchanged version that reflects the state of the database before the transaction started.

Changes are written to new copies of the database pages, leaving the shadow pages unchanged until the transaction is committed. This ensures the database can recover to a consistent state without requiring log processing.

---

### Steps in Shadow Paging

1. **Before Modifying a Page:**
    - Create a **shadow copy** of the current page. The shadow copy serves as a backup of the page's original state.
  2. **Write Changes to a New Page:**
    - Perform all modifications on a new copy of the page rather than directly updating the original.
  3. **Commit the Changes:**
    - Update the **page table** to point to the newly modified pages.
    - Once committed, the shadow pages are discarded, and the modified pages become the new current pages.
- 

### Example of Shadow Paging

#### Scenario:

- A database contains two pages, P1 and P2.

#### 1. Initial State:

- Page table points to:

**Page Table: [P1 -> Address A1, P2 -> Address A2]**

**Shadow Pages: [P1 -> Address A1, P2 -> Address A2]**

**2. Transaction Starts:**

- Modify P1:
  - Create a shadow copy of P1 at Address A1.
  - Write changes to a new page at Address A3.

**3. Page Table Update on Commit:**

- Update the page table to reflect the new address for P1:

**Page Table: [P1 -> Address A3, P2 -> Address A2]**

- Discard the shadow copy of P1 at Address A1.

**4. Crash Recovery:**

- If a crash occurs before committing:
  - The page table still points to the shadow pages ([P1 -> Address A1, P2 -> Address A2]), ensuring a consistent state.

---

**Advantages of Shadow Paging**

**1. No Undo Logging Required:**

- Since shadow pages remain unchanged until a transaction is committed, there is no need to log before-images for rollback.

**2. Crash Recovery Without Log Processing:**

- The database can revert to the shadow pages after a crash, eliminating the need to scan logs.

**3. Immediate Consistency:**

- Only committed changes are reflected in the active database.

**4. Simplicity:**

- Avoids the complexities of maintaining detailed logs for redo and undo operations.

---

**Limitations of Shadow Paging**

**1. Overhead of Maintaining Shadow Copies:**

- Requires additional storage to maintain copies of the pages being modified.

**2. Increased Storage Requirements:**

- Every transaction requires new copies of the modified pages until the commit.
- 3. **Complex Page Table Management:**
  - Updating and maintaining the page table during transactions adds complexity.
- 4. **Write Amplification:**
  - Frequent modifications may lead to increased I/O operations due to repeated copying of pages.
- 5. **Concurrency Issues:**
  - Shadow paging can complicate concurrency control in multi-user environments.

---

### Comparison with Log-Based Recovery

| Aspect               | Shadow Paging                                      | Log-Based Recovery                     |
|----------------------|----------------------------------------------------|----------------------------------------|
| Recovery             | No log processing required.                        | Requires undo and redo log processing. |
| Storage Requirements | Higher due to shadow copies.                       | Lower, as logs are compact.            |
| Performance          | Slower for frequent updates (write amplification). | Faster for incremental updates.        |
| Complexity           | Simpler recovery process.                          | More complex due to log management.    |

---

### Use Cases

1. **Static Databases:**
  - Shadow paging is ideal for systems with infrequent updates, such as read-heavy environments or archival systems.
2. **Embedded Systems:**
  - Used in lightweight database systems where simplicity and crash recovery are prioritized over performance (e.g., mobile devices).

---

### Real-World Example

- **Berkeley DB:**
  - Uses a variant of shadow paging for transactional support in embedded systems.
- **File Systems:**
  - Some file systems, like ZFS, employ shadow paging-like techniques to maintain consistency.

---

## Summary

- **Definition:**
  - Shadow paging creates stable shadow copies of database pages, ensuring recovery without log processing.
- **Steps:**
  - Create shadow copies, modify new pages, and update the page table upon commit.
- **Advantages:**
  - Eliminates the need for undo logging, simplifies recovery, and provides immediate consistency.
- **Limitations:**
  - Higher storage overhead, write amplification, and complexity in managing the page table.

Shadow paging is a powerful crash recovery technique, especially in systems where simplicity and consistency outweigh performance considerations.